

VAULT-TEC

MEDIA OPTIMIZER

Dépendances, performances & bonnes pratiques
Dependencies, performance & best practices

Complément technique à la documentation v1.4.1 — couvre la **v1.8.2**
Moteur libvips · GIF (gifsicle) · AVIF · Coût/bénéfice réel modélisé + mesures (13 graphiques)
Document **bilingue FR / EN** — Extension MediaWiki — Archives de Vault-Tec

FR · Partie française ci-dessous · **EN** · English version follows (same content)

1. À propos de ce document

Ce document **complète** le guide d'installation et d'administration v1.4.1 ; il ne le répète pas. Il se concentre sur quatre sujets : les **nouveautés** depuis la 1.4.1, l'**installation des dépendances**, des **benchmarks honnêtes (modèle coût/bénéfice du wiki réel + mesures)**, et les **bonnes pratiques anti-ralentissement**. Pour la configuration générale, les pages spéciales et le dépannage, reportez-vous au guide v1.4.1 ; pour le détail des paramètres récents, à `docs/NEW_FEATURES.md` et `CHANGELOG.md`.

Sans doublon. Les commandes de configuration ne sont rappelées que lorsqu'elles sont indispensables à la compréhension d'une dépendance ou d'une bonne pratique.

2. Nouveautés depuis la v1.4.1

Version	Apport
1.7.0	Moteur d'image libvips optionnel (vitesse/mémoire à l'échelle) ; mode gros wiki (<code>optimizeImages.php</code> , <code>purgeQueue.php</code> , <code>UseJobQueue</code>) ; <code>OnDemandThumbLimit</code> ; portabilité des statistiques (SQLite/PostgreSQL) ; écritures atomiques.
1.8.0	Optimisation GIF sans perte via le binaire externe <code>gifsicle</code> (-03), en 1 ^{re} passe, sûre pour l'animation (jamais de redimensionnement / d'altération des frames / de la cadence / de la boucle). Option <code>--format</code> du script de rattrapage (ex. <code>--format=gif</code>). Désactivé par défaut, opt-in.
1.8.1	Correctifs robustesse : resynchronisation des métadonnées MediaWiki (<code>img_sha1/img_size</code>) après la 1 ^{re} passe en place ; les GIF animés ne sont jamais figés — sous GD (qui ne décode que la 1 ^{re} frame) le WebP/AVIF est refusé et le GIF animé d'origine conservé ; les GIF statiques se convertissent enfin en WebP/AVIF sous GD (images à palette).
<i>exp.</i>	AVIF expérimental (branche dédiée) : génère l'AVIF en plus du WebP et le propose <i>avant</i> le WebP dans <code><picture></code> . Sonde réelle de capacité AV1 → repli propre sur WebP si l'encodeur ne sait pas faire d'AV1.

3. Installation des dépendances

L'extension fonctionne avec un minimum : une bibliothèque d'image avec support WebP. Tout le reste est **optionnel** et n'apporte qu'un bénéfice ciblé.

Dépendance	Rôle	Obligatoire ?
Imagick <i>ou</i> GD (avec WebP)	1 ^{re} passe + encodage WebP	Oui (l'une des deux)
libvips (<i>vips</i>)	Moteur alternatif rapide/léger (§4)	Non — opt-in
gifsicle	1 ^{re} passe GIF sans perte (animation sûre)	Non — gain GIF
zopflipng	2 ^e passe PNG sans perte (compression max)	Non — gain disque
oxipng	2 ^e passe PNG sans perte (rapide)	Non — gain disque
libheif+AV1 / <code>imageavif()</code>	AVIF expérimental	Non — expérimental

3.1 Bibliothèque d'image : Imagick (recommandé) ou GD

```
# Debian / Ubuntu
sudo apt install php-imagick      # recommandé (vraie transparence/ICC, vrai WebP lossless)
sudo apt install php-gd           # repli, souvent déjà présent

# RHEL / Alma / Rocky / Fedora
sudo dnf install php-imagick php-gd
sudo systemctl restart php-fpm apache2 2>/dev/null || true
```

Vérifiez le **support WebP** (sans lui, aucune génération possible) :

```
php -r 'echo extension_loaded("imagick")&&in_array("WEBP",Imagick::queryFormats())?"Imagick+WebP OK\n":"";'
php -r '$i=gd_info();echo !empty($i["WebP Support"])?"GD+WebP OK\n":"GD sans WebP\n";'
```

GD ≠ vrai WebP/AVIF sans perte. GD n'a pas de mode *loss/less* réel (il approxime en qualité 100) : sur une capture d'UI à aplats, GD produit un WebP **~2,7× plus gros** qu'Imagick/libvips (mesuré : 19 Ko vs 6,9 Ko, §4.4). Pour les PNG nets, préférez Imagick ou libvips.

3.2 Optimisation GIF : gifsicle

```
# Debian / Ubuntu
sudo apt install gifsicle
# RHEL / Alma / Rocky / Fedora (EPEL)
sudo dnf install gifsicle
gifsicle --version | head -1
```

Activation (off par défaut) : `$wgVaultTecMediaOptimizerGifsicleEnabled = true;` — niveau via `...GifsicleLevel` (1-3, défaut 3), chemin via `...GifsicleBinary`. **Strictement sans perte et sûr pour l'animation** : seul `-0{niveau}` est passé (jamais de `resize` / `lossy` / `colors`). Si `gifsicle` manque, c'est un no-op inoffensif — la génération WebP continue.

3.3 Seconde passe PNG : zopflipng (max) ou oxipng (rapide)

```
sudo apt install zopfli      # /usr/bin/zopflipng (Debian/Ubuntu) – compression max, LENT
cargo install oxipng        # oxipng (Rust, multi-thread, rapide) si non empaqueté
```

Piège `open_basedir` (mutualisé/Plesk). Si PHP est restreint, un binaire dans `/usr/bin` reste invisible côté web. **Règle d'or** : copiez le binaire *dans* l'arborescence du wiki (ex. `httpdocs/bin/oxipng`), `chmod 755`, propriétaire = utilisateur PHP, puis configurez le **chemin absolu**. Idem pour `vips`, `zopflipng` et `gifsicle`.

3.4 Moteur libvips (optionnel — voir §4 pour le gain)

```
# Debian/Ubuntu : sudo apt install libvips-tools | RHEL/Fedora : sudo dnf install vips-tools
# Alpine : apk add vips-tools | macOS (dev) : brew install vips
vips --version ; vips -l | grep -i wepssave
```

Activation : `$wgVaultTecMediaOptimizerImageEngine = 'vips';` (+ `...VipsBinary` si besoin). Si le binaire est inutilisable, l'extension **retombe automatiquement** sur Imagick/GD.

3.5 AVIF (expérimental)

L'AVIF exige un encodeur **AV1** : Imagick compilé avec `libheif+AV1`, PHP-GD avec `imageavif()`, ou libvips avec `heifsave+AV1`. **Beaucoup de paquets exposent `heifsave` pour le HEIC sans AV1** — l'extension le détecte et sert alors le WebP. Test réel de la capacité AV1 de vips :

```
vips black /tmp/p.png 16 16 && vips heifsave /tmp/p.png /tmp/p.avif --compression av1 \
&& echo "AV1 OK" || echo "AV1 indisponible"
```

4. Benchmarks — ce que ça coûte, ce que ça rapporte

4.1 Hypothèses & méthode

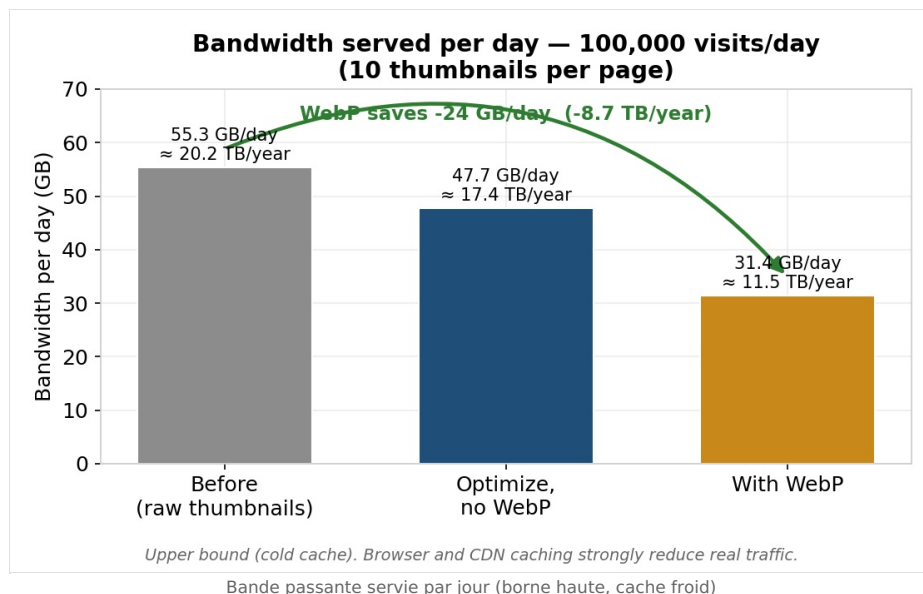
Deux familles de chiffres. **(A)** un **modèle coût/bénéfice** du wiki réel, recalculé depuis des hypothèses explicites ; **(B)** des **mesures réelles** prises sur la machine de test (PHP 8.4, libvips 8.15.1, ImageMagick 6.9.12, gifsicle 1.94, zopflipng, PHP-GD ; **un seul cœur**). Tout est reproductible via `docs/benchmarks/model.py` et `tests/integration/pipeline.php`.

Hypothèses du modèle (unités décimales ; 365 j/an) :

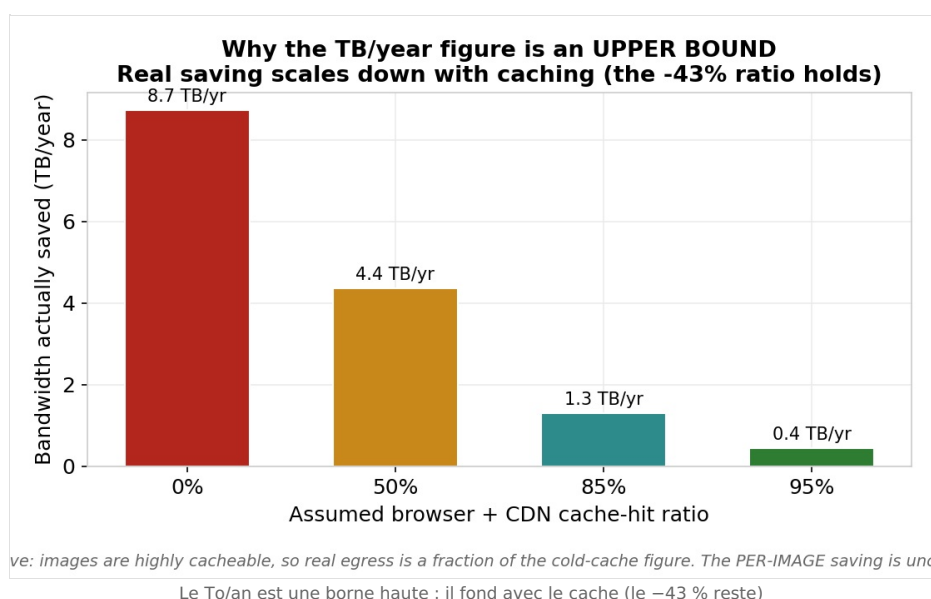
- Originaux du wiki : **52,62 Go** de PNG+JPEG (chiffre réel).
- Trafic en **borne haute (cache froid)** : **100 000 vues/jour, 10 miniatures/page** → 1,0 M miniatures servies/jour. Le cache navigateur + CDN réduit fortement le trafic réel.
- Miniature servie moyenne **55,3 Ko** ; optimisation sans perte de la miniature −13,7 % ; WebP de la miniature −34 % (≈ **−43 %** vs brut).
- Recompression sans perte des originaux — 3 scénarios : **−7 % / −16 % / −24 %**.
- Copies WebP des originaux ≈ **70 %** de l'original optimisé, stockées **en plus**.

Pourquoi ce cadrage ? Une estimation antérieure confondait *compression par fichier* et *bande passante servie*. Les pages servent des **miniatures** (déjà petites), pas les originaux, et le WebP **ajoute** du disque puisqu'il s'ajoute aux originaux. Les figures séparent ces deux choses.

4.2 Bande passante servie — le vrai bénéfice

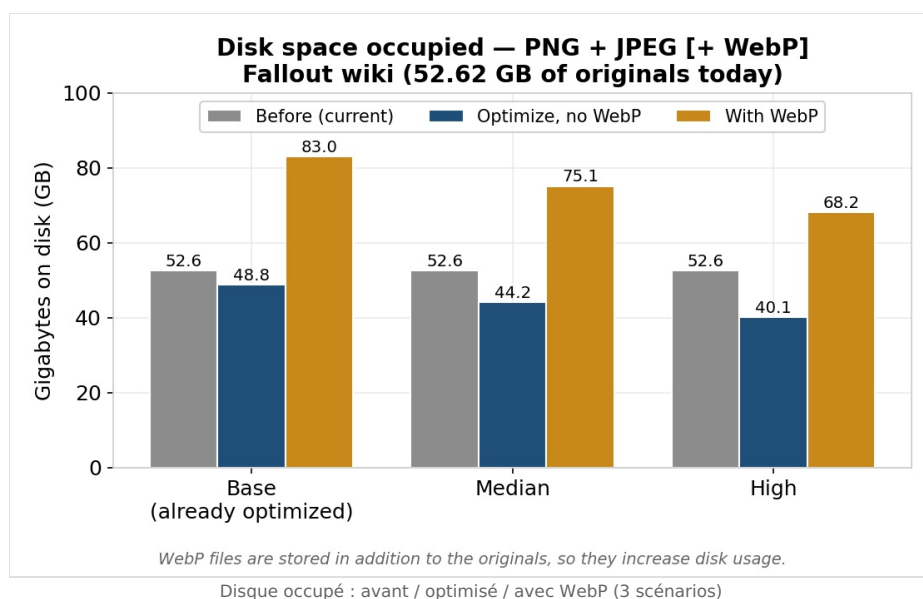


Au niveau des **miniatures réellement servies**, le WebP fait passer le trafic de **~55 à ~31 Go/jour (-43 %)**, soit **~8,7 To/an — en borne haute (cache froid)**.

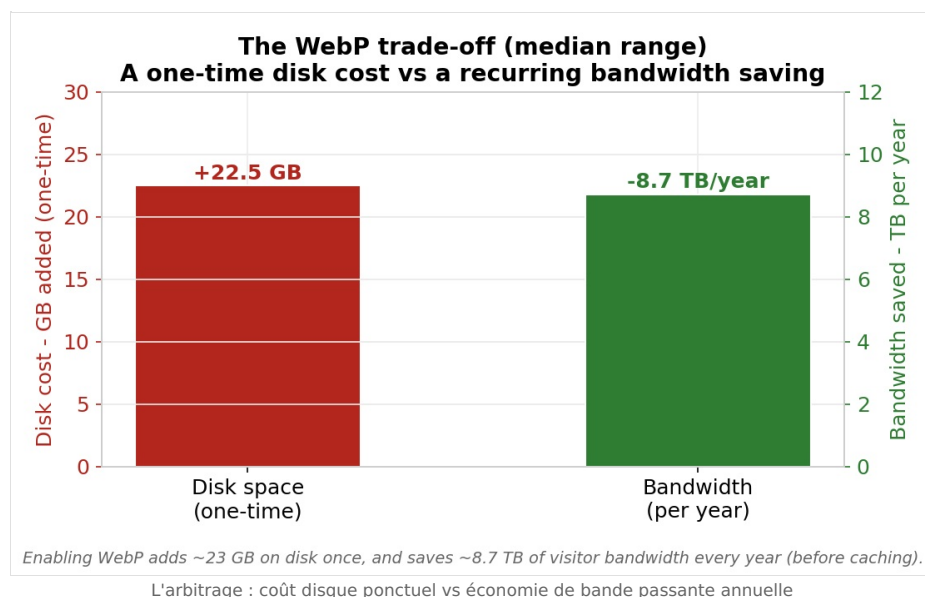
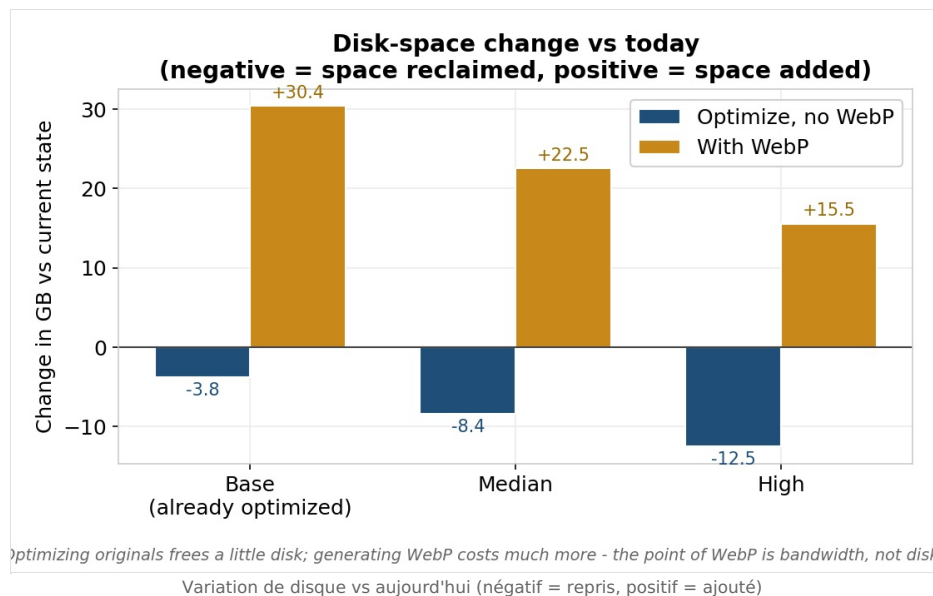


Le **ratio -43 % est robuste** ; le **To/an absolu est une borne haute** : les images sont très « cachables », donc avec un CDN/navigateur l'économie réelle n'est qu'une fraction de ce chiffre.

4.3 Disque : le WebP en AJOUTE

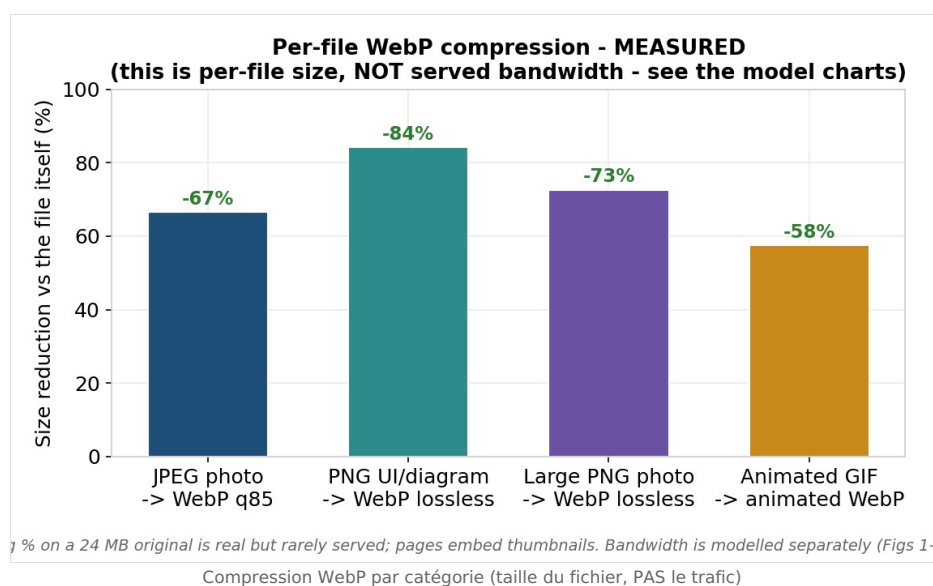


Les WebP sont stockés **en plus** des originaux : le disque **monte** (52,6 → ~75 Go au médian). Optimiser les originaux ne reprend qu'un peu (-4 à -12 Go).

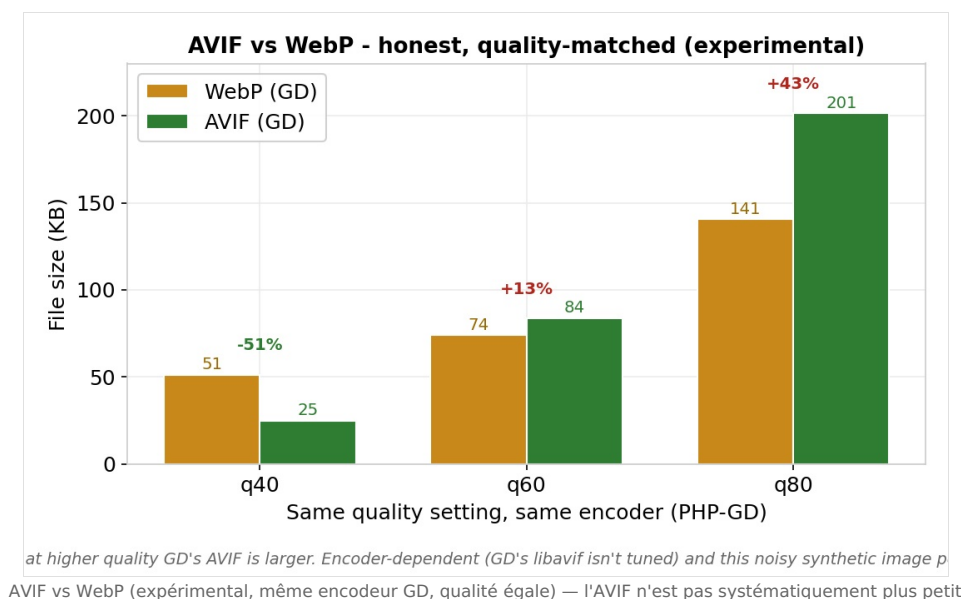
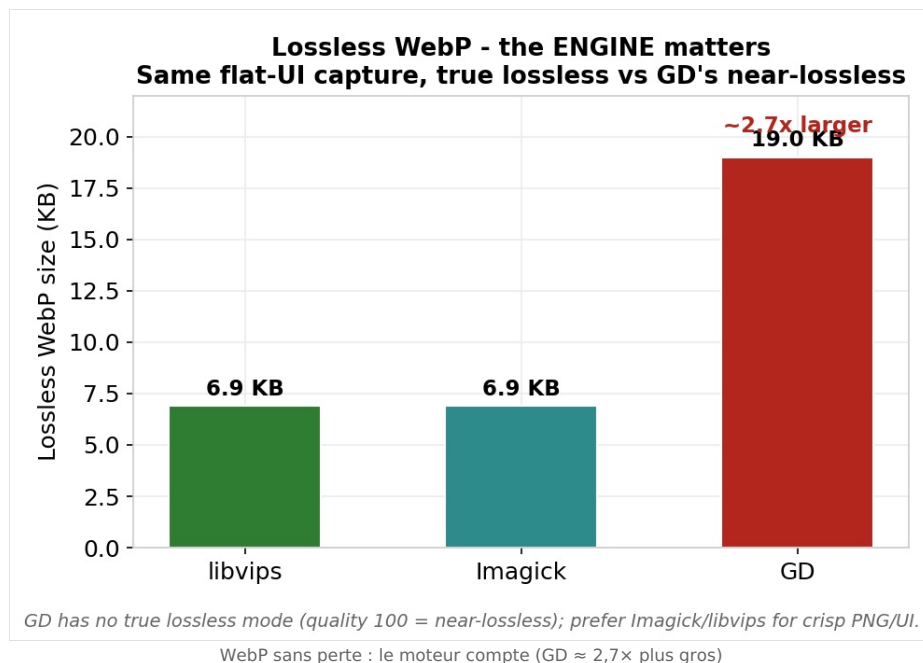


L'arbitrage honnête : un **coût disque ponctuel** (~+22 Go au médian) contre une **économie de bande passante récurrente**. Le WebP est un levier de **bande passante**, pas de disque.

4.4 Compression par fichier (mesurée) — ≠ bande passante

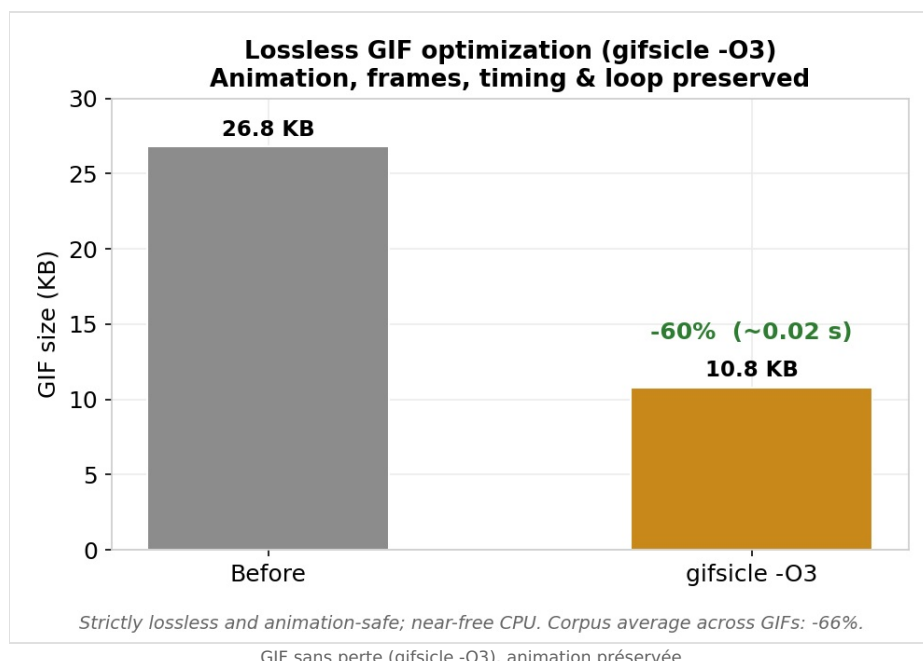


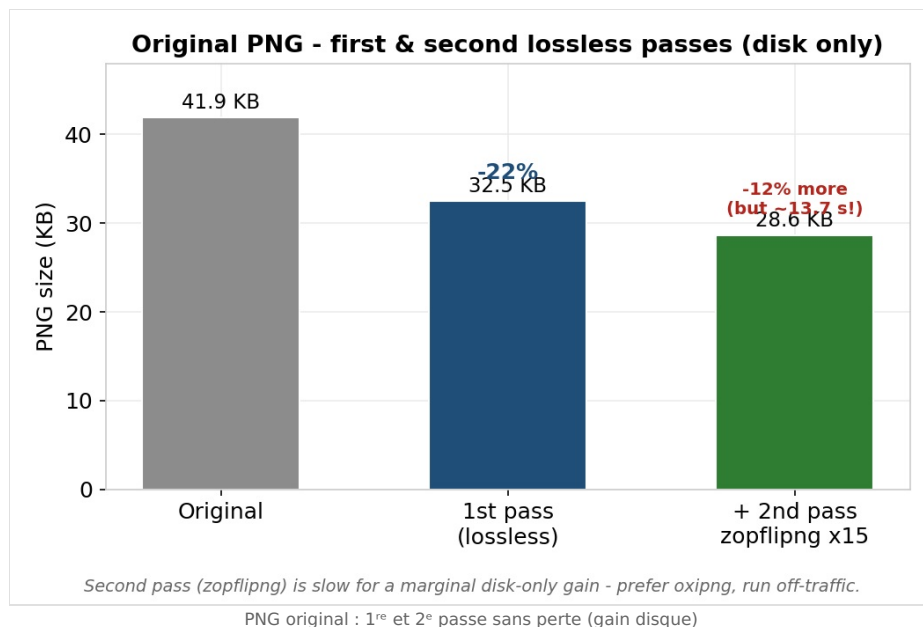
Ces gains sont réels mais portent sur le **fichier lui-même** : un -73 % sur un PNG de 24 Mo est spectaculaire mais **rarement servi** (on sert sa miniature). D'où la modélisation séparée de la bande passante (§4.2).



Le **moteur compte** : GD n'a pas de vrai lossless. À qualité égale, l'**AVIF n'est pas systématiquement plus petit** que le WebP : cela dépend du contenu et de l'encodeur (avec GD, il ne gagne qu'en basse qualité). De plus l'encodeur AV1 manquait côté vips dans l'env de test (repli WebP automatique). D'où le statut expérimental.

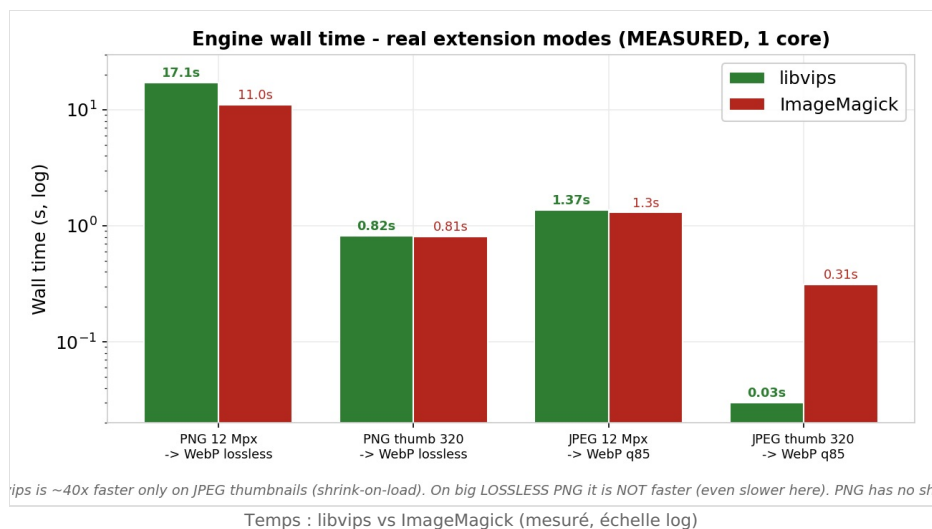
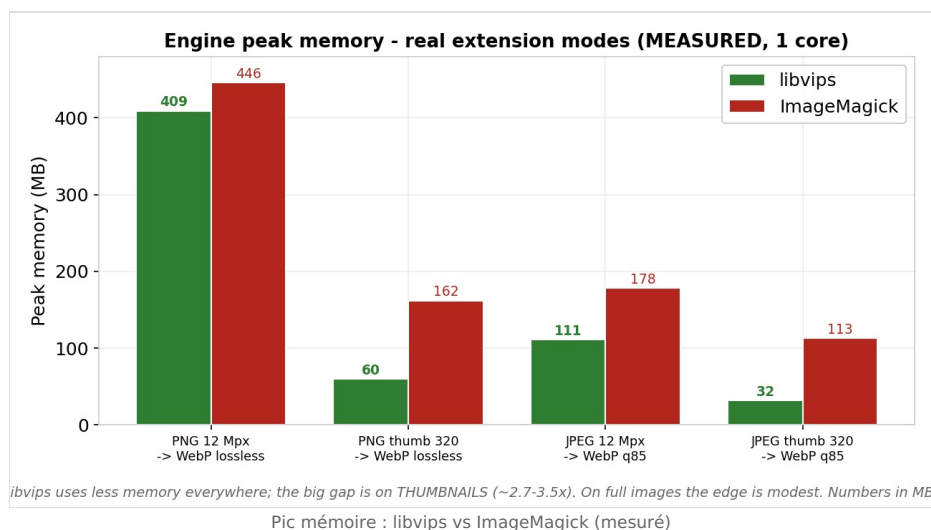
4.5 GIF & 2^e passe PNG (disque)



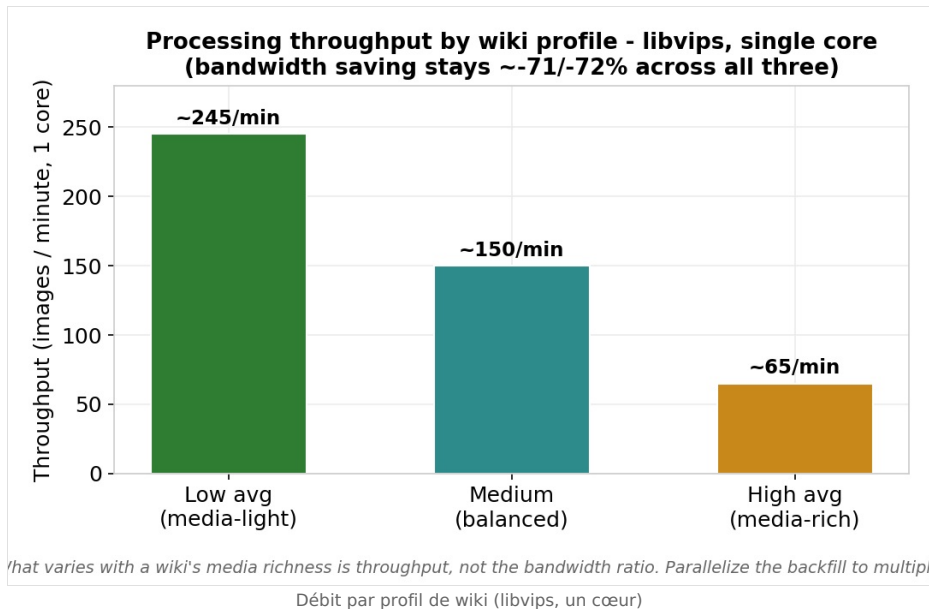


gifsicle : -60 à -66 % sur les GIF, sans perte ni atteinte à l'animation, pour ~20 ms. La **2^e passe PNG (zopflipng)** ne grappille que ~-12 % de disque... en ~14 s : hors trafic, ou préférez oxipng.

4.6 Moteur : mémoire, vitesse, débit



Lecture honnête (mesures aux réglages réels de l'extension : PNG → WebP lossless, JPEG → q85). libvips utilise **moins de mémoire partout**, surtout sur les **miniatures** (~2,7-3,5x). Côté **vitesse**, l'avantage n'est **pas** universel : il est net sur les **miniatures JPEG** (~40x, grâce au *shrink-on-load*), mais sur un **gros PNG lossless** libvips n'est **pas plus rapide** (il est même un peu plus lent : 17 s vs 11 s), car le PNG n'a pas de décodage à échelle réduite. Bilan : vips fait surtout gagner de la **mémoire** (et du temps sur les miniatures JPEG).



Le **débit** varie selon la richesse média (~245 à ~65 images/min sur un cœur) ; le ratio de bande passante reste ~71/-72 %. Parallélisez le backfill pour multiplier le débit.

5. Bonnes pratiques anti-ralentissement

5.1 D'où vient le risque ?

- **File de tâches sur les requêtes web.** Par défaut, MediaWiki exécute des jobs en fin de requête (`$wgJobRunRate`) : ce sont alors vos **visiteurs** qui « paient » l'encodage.
- **Génération WebP à la volée au rendu.** Le premier affichage d'une page froide encode les WebP manquants des miniatures qu'elle utilise (borné par `OnDemandThumbLimit`).
- **2^e passe synchrone (si activée).** zopflipng sur une miniature peut coûter plusieurs centaines de ms à plusieurs secondes (cf. §4.3).

5.2 Première installation (surtout gros wiki)

Option A — tout en CLI (recommandé gros wiki)	Option B — par la file, maîtrisée
<pre>// LocalSettings.php \$wgVaultTecMediaOptimizerUseJobQueue = false;</pre> Puis dans un <code>screen</code> : <pre>php maintenance/run.php \ .../optimizeImages.php --batch=200</pre> Hors file, sans impacter les visiteurs. Reprenable (<code>--start</code>), filtrable (<code>--format=gif</code>).	<pre>// LocalSettings.php — 0, pas plus ! \$wgJobRunRate = 0;</pre> Puis videz la file en SSH : <pre>php maintenance/run.php runJobs.php \ --type VaultTecMediaOptimizerOptimizeImage</pre> Jobs résiduels : <code>purgeQueue.php</code>.

Ne montez pas `$wgJobRunRate` : cela **aggrave** le ralentissement (plus de jobs lourds sur les requêtes visiteurs). Pour un traitement lourd, la bonne valeur est **0**, combinée à un `runJobs.php` en SSH — ou le script CLI dédié. **Le 2^e passage zopflipng des miniatures** passe lui aussi par un **job** (depuis la 1.8.2) : même règle — traitez la file hors requête pour qu'aucun visiteur ne paie ce temps.

- **Choisissez le moteur selon le volume.** Grosse photothèque → **libvips** (mémoire/vitesse, §4.4).
- **Réglez `OnDemandThumbLimit`** (défaut 5) : bas = premiers rendus plus légers ; 0 = tout par le backfill.
- **Purgez les caches À LA FIN** (wiki + CDN), pas pendant.

5.3 Usage courant

- **GIF** : activez `gifsicle` — gain franc (~66 %), coût négligeable, animation préservée.
- **2^e passe PNG** : facultative et de préférence **oxipng** ; gardez zopflipng pour une finition hors trafic.
- **En-têtes de cache HTTP longs** sur `images_webp/` (et `images_avif/`) : levier majeur souvent oublié pour réduire réellement la bande passante.
- **Ne mélangez pas file et CLI** sur le même corpus (écritures atomiques = pas de corruption, mais travail en double).
- **Droits** : lancez les scripts sous l'utilisateur PHP, jamais en `root`.

1. About this document

This document **complements** the v1.4.1 installation and administration guide; it does not repeat it. It focuses on four topics: **what's new** since 1.4.1, **installing the dependencies**, **honest benchmarks (real-wiki cost/benefit model + measurements)**, and **best practices to avoid slowing the wiki down**. For general configuration, special pages and troubleshooting, see the v1.4.1 guide; for recent settings in detail, see `docs/NEW_FEATURES.md` and `CHANGELOG.md`.

No duplication. Configuration commands are repeated here only when essential to understand a dependency or a best practice.

2. What's new since v1.4.1

Version	What it adds
1.7.0	Optional libvips image engine (speed/memory at scale); large-wiki mode (<code>optimizeImages.php</code> , <code>purgeQueue.php</code> , <code>UseJobQueue</code>); <code>OnDemandThumbLimit</code> ; portable statistics (SQLite/PostgreSQL); atomic writes.
1.8.0	Lossless GIF optimization via the external <code>gifsicle</code> binary (<code>-03</code>), as a first pass, animation-safe (never resizes / never alters frames / timing / loop). <code>--format</code> filter for the catch-up script (e.g. <code>--format=gif</code>). Off by default, opt-in.
1.8.1	Robustness fixes: MediaWiki metadata (<code>img_sha1/img_size</code>) refreshed after the in-place first pass; animated GIFs are never frozen — under GD (which decodes only the first frame) the WebP/AVIF is refused and the original animated GIF kept; still GIFs finally convert to WebP/AVIF under GD (palette images).
<i>exp.</i>	Experimental AVIF (dedicated branch): generates AVIF alongside WebP and offers it <i>before</i> WebP inside <code><picture></code> . Real AV1-capability probe → clean fallback to WebP when the encoder cannot do AV1.

3. Installing the dependencies

The extension runs with a bare minimum: an image library with WebP support. Everything else is **optional** and only adds a targeted benefit.

Dependency	Role	Required?
Imagick <i>or</i> GD (with WebP)	First pass + WebP encoding	Yes (either one)
libvips (<code>vips</code>)	Fast/light alternative engine (\$4)	No — opt-in
gifsicle	Lossless GIF first pass (animation-safe)	No — GIF gain
zopflipng	2nd-pass lossless PNG (max compression)	No — disk gain
oxipng	2nd-pass lossless PNG (fast)	No — disk gain
libheif+AV1 / <code>imageavif()</code>	Experimental AVIF	No — experimental

3.1 Image library: Imagick (recommended) or GD

```
# Debian / Ubuntu
sudo apt install php-imagick      # recommended (true transparency/ICC, true lossless WebP)
sudo apt install php-gd           # fallback, often already present
# RHEL / Alma / Rocky / Fedora
sudo dnf install php-imagick php-gd
sudo systemctl restart php-fpm apache2 2>/dev/null || true
```

Check **WebP support** (without it, no generation is possible):

```
php -r 'echo extension_loaded("imagick")&&in_array("WEBP",Imagick::queryFormats())?"Imagick+WebP OK\n":"";'
php -r '$i=gd_info();echo !empty($i["WebP Support"])?"GD+WebP OK\n":"GD without WebP\n";'
```

GD is not true lossless WebP/AVIF. GD has no real *loss/ess* mode (it approximates with quality 100): on a flat UI capture, GD produces a WebP **~2.7× larger** than Imagick/libvips (measured: 19 KB vs 6.9 KB, \$4.4). For crisp PNGs, prefer Imagick or libvips.

3.2 GIF optimization: gifsicle

```
# Debian / Ubuntu
sudo apt install gifsicle
# RHEL / Alma / Rocky / Fedora (EPEL)
sudo dnf install gifsicle
gifsicle --version | head -1
```

Enable (off by default): `$wgVaultTecMediaOptimizerGifsicleEnabled = true;` — level via `...GifsicleLevel` (1–3, default 3), path via `...`

GifsicleBinary. **Strictly lossless and animation-safe:** only `-O{level}` is passed (never `resize` / `lossy` / `colors`). If `gifsicle` is missing it is a harmless no-op — WebP generation still proceeds.

3.3 Second-pass PNG: zopflipng (max) or oxipng (fast)

```
sudo apt install zopfli      # /usr/bin/zopflipng (Debian/Ubuntu) – max compression, SLOW
cargo install oxipng        # oxipng (Rust, multi-threaded, fast) when not packaged
```

open_basedir trap (shared hosting/Plesk). If PHP is restricted, a binary in `/usr/bin` stays invisible to the web side. **Golden rule:** copy the binary *inside* the wiki tree (e.g. `httpdocs/bin/oxipng`), `chmod 755`, owner = PHP user, then configure the **absolute path**. Same for `vips`, `zopflipng` and `gifsicle`.

3.4 libvips engine (optional — see §4 for the gain)

```
# Debian/Ubuntu: sudo apt install libvips-tools | RHEL/Fedora: sudo dnf install vips-tools
# Alpine: apk add vips-tools                  | macOS (dev): brew install vips
vips --version ; vips -l | grep -i webspave
```

Enable: `$wgVaultTecMediaOptimizerImageEngine = 'vips';` (+ `...VipsBinary` if needed). If the binary is unusable, the extension **falls back automatically** to Imagick/GD.

3.5 AVIF (experimental)

AVIF needs an **AV1** encoder: Imagick built with `libheif+AV1`, PHP-GD with `imageavif()`, or libvips with `heifsave+AV1`. **Many packages expose `heifsave` for HEIC without AV1** — the extension detects this and serves WebP instead. Real AV1-capability test for vips:

```
vips black /tmp/p.png 16 16 && vips heifsave /tmp/p.png /tmp/p.avif --compression av1 \
&& echo "AV1 OK" || echo "AV1 unavailable"
```

4. Benchmarks — what it costs, what it saves

4.1 Assumptions & method

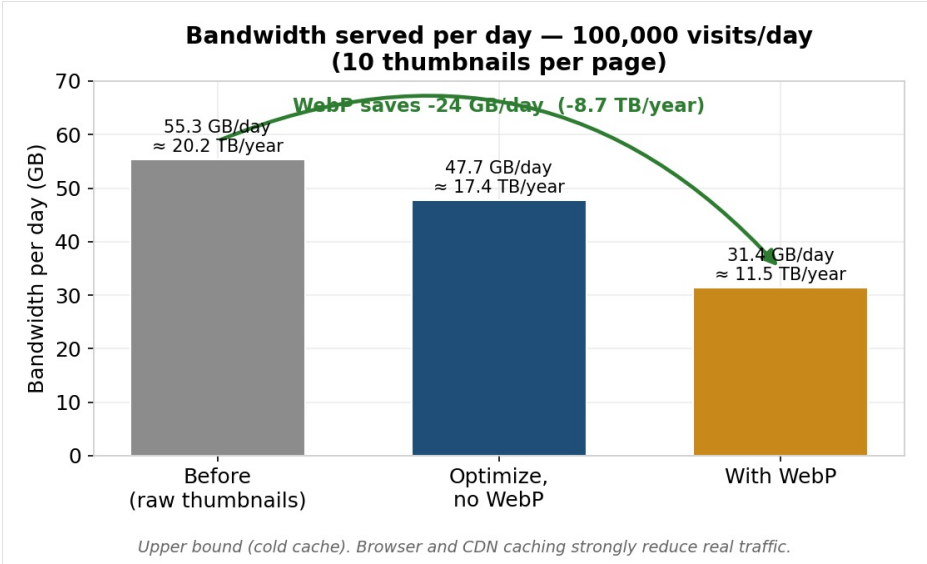
Two families of figures. **(A)** a **cost/benefit model** of the real wiki, recomputed from explicit assumptions; **(B)** **real measurements** on the test box (PHP 8.4, libvips 8.15.1, ImageMagick 6.9.12, gifsicle 1.94, zopflipng, PHP-GD; **single core**). Everything is reproducible via `docs/benchmarks/model.py` and `tests/integration/pipeline.php`.

Model assumptions (decimal units; 365 d/yr):

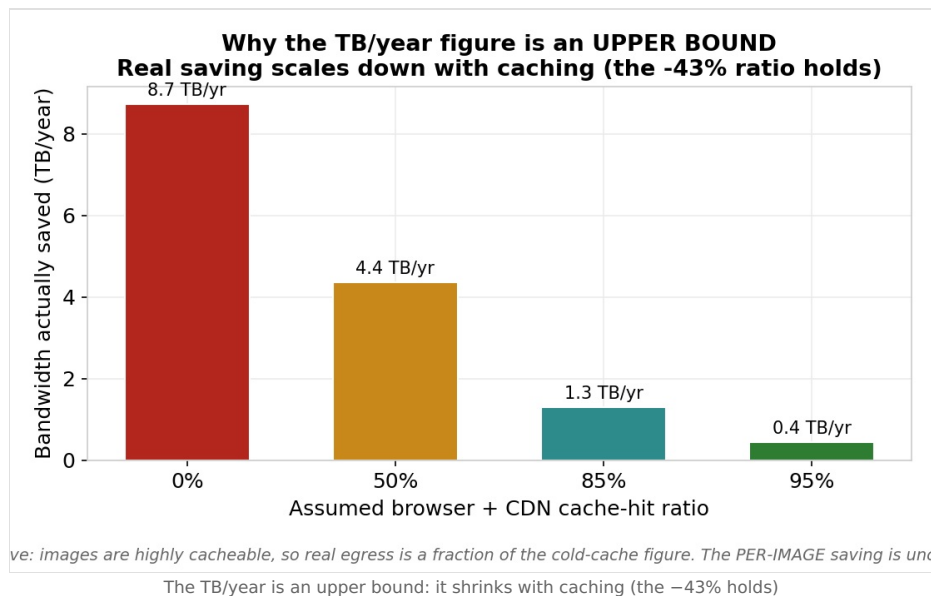
- Wiki originals: **52.62 GB** of PNG+JPEG (real figure).
- Traffic **upper bound (cold cache): 100,000 views/day, 10 thumbnails/page** → 1.0 M thumbnail-serves/day. Browser + CDN caching strongly reduce real traffic.
- Average served thumbnail **55.3 KB**; lossless thumbnail optimization −13.7%; WebP of the thumbnail −34% (≈ **−43%** vs raw).
- Lossless original recompression — 3 scenarios: **−7%** / **−16%** / **−24%**.
- WebP copies of originals ≈ **70%** of the optimized original, stored **in addition**.

Why this framing? An earlier estimate conflated *per-file compression* with *served bandwidth*. Pages serve **thumbnails** (already small), not originals, and WebP adds disk since it sits alongside the originals. The figures separate the two.

4.2 Served bandwidth — the real benefit

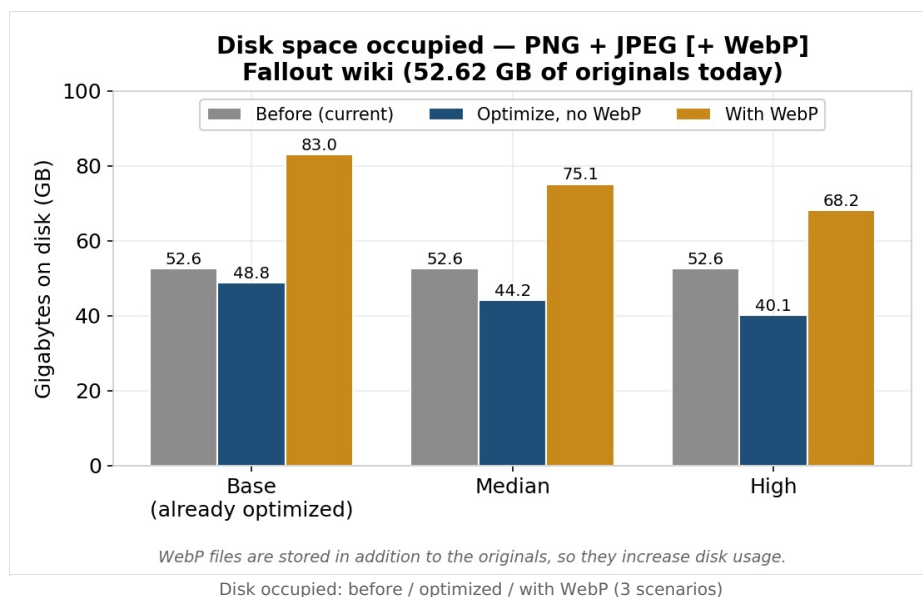


At the level of **thumbnails actually served**, WebP takes traffic from **~55 to ~31 GB/day (-43%)**, i.e. **~8.7 TB/year** — as a **cold-cache upper bound**.

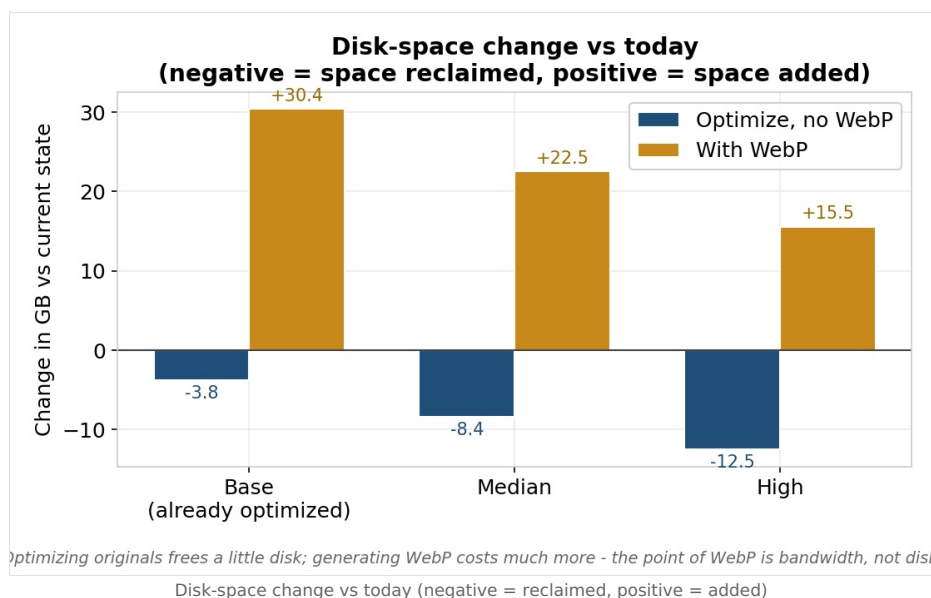


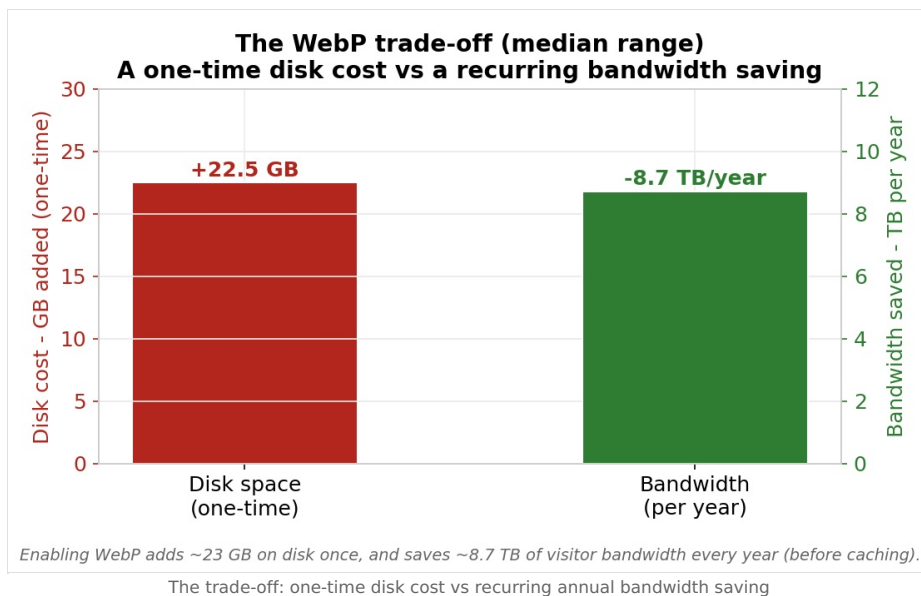
The **-43% ratio is robust**; the **absolute TB/year is an upper bound**: images are highly cacheable, so with a CDN/browser the real saving is a fraction of that figure.

4.3 Disk: WebP ADDS to it



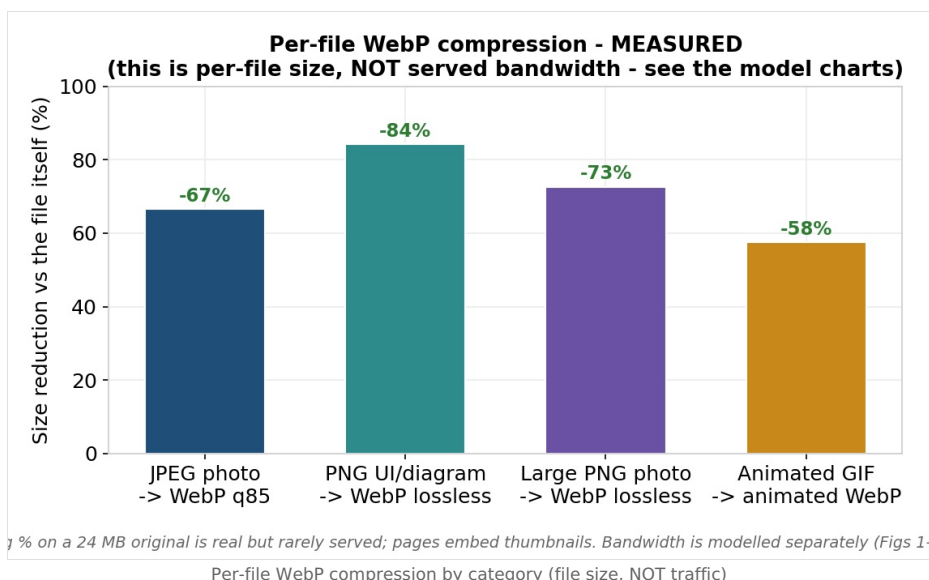
WebP files are stored **in addition** to the originals: disk **goes up** (52.6 → ~75 GB median). Optimizing originals only reclaims a little (-4 to -12 GB).



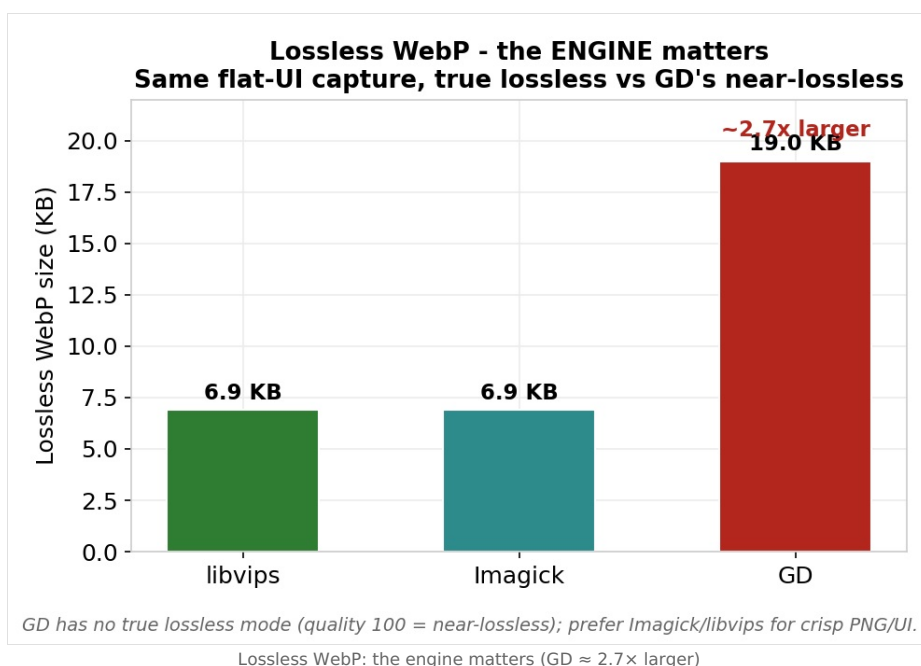


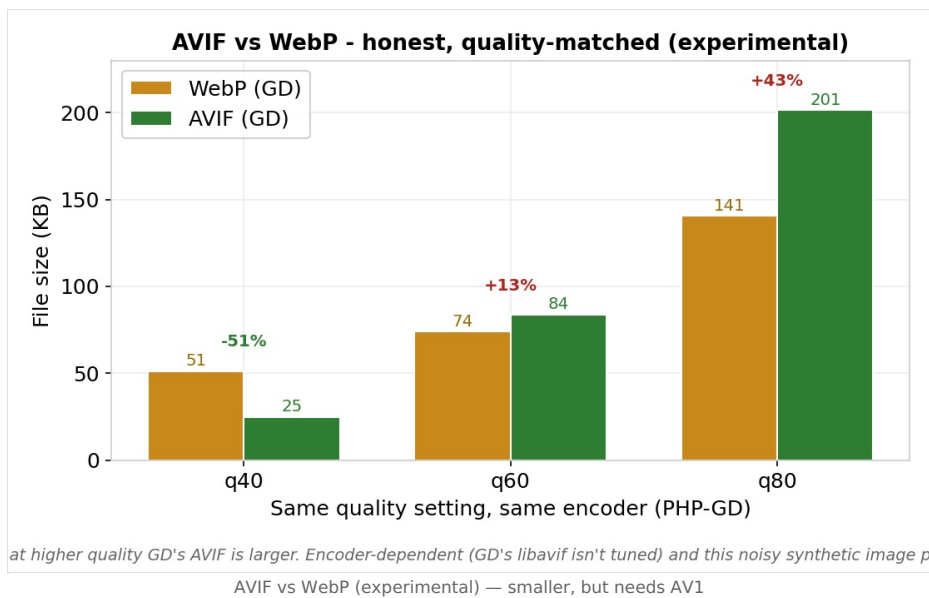
The honest trade-off: a **one-time disk cost** (~+22 GB median) against a **recurring bandwidth saving**. WebP is a **bandwidth** lever, not a disk one.

4.4 Per-file compression (measured) — ≠ bandwidth



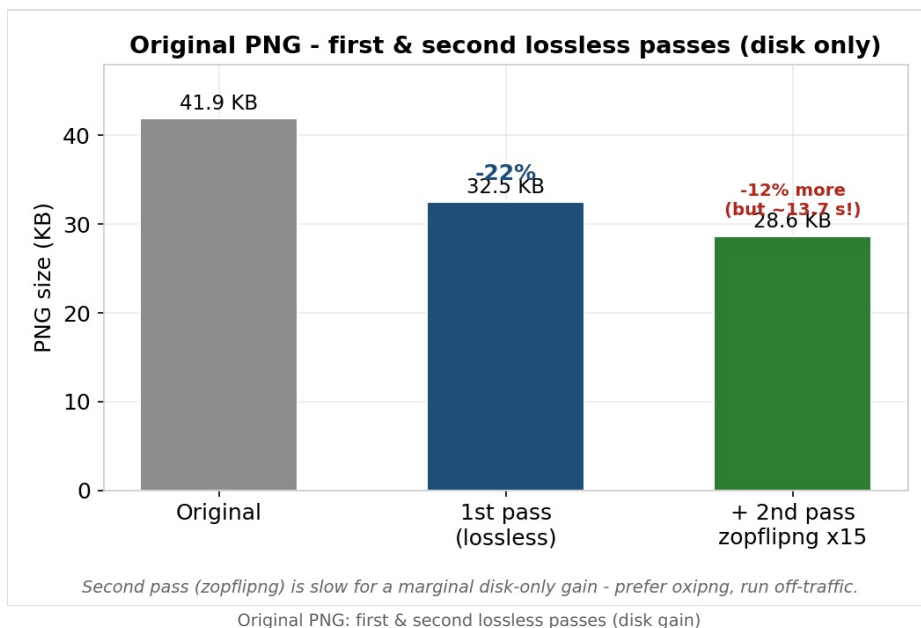
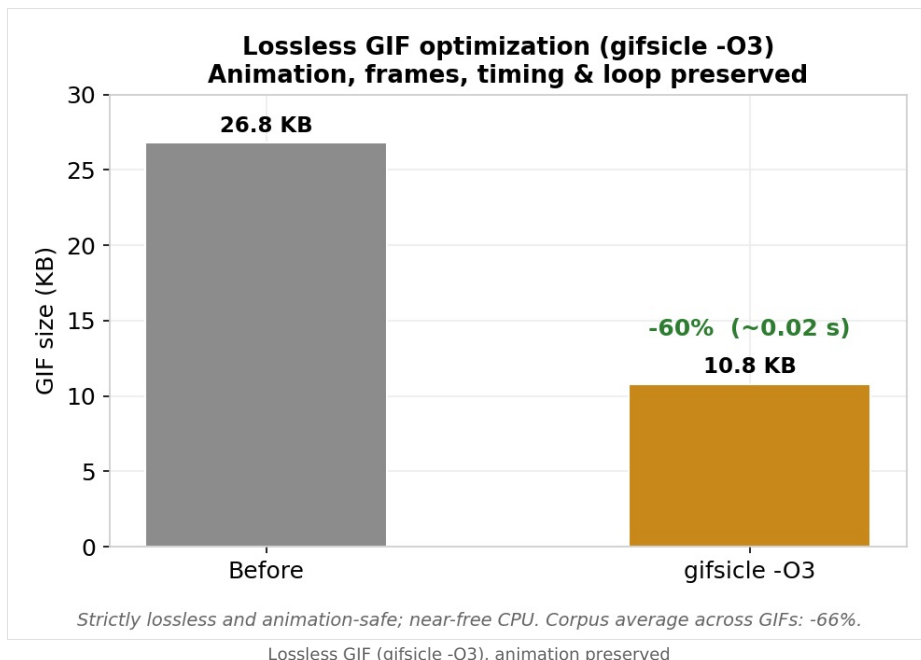
These gains are real but apply to the **file itself**: –73% on a 24 MB PNG is striking but **rarely served** (its thumbnail is). Hence the separate bandwidth model (§4.2).





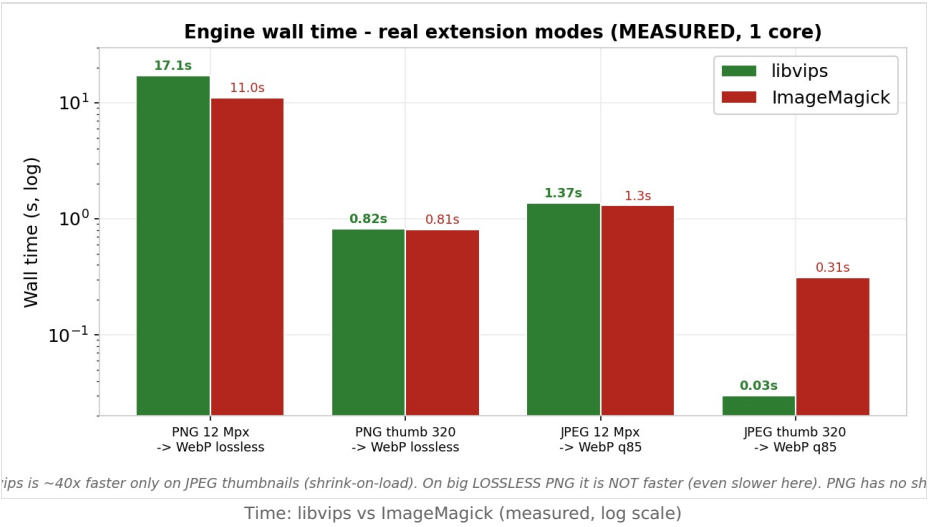
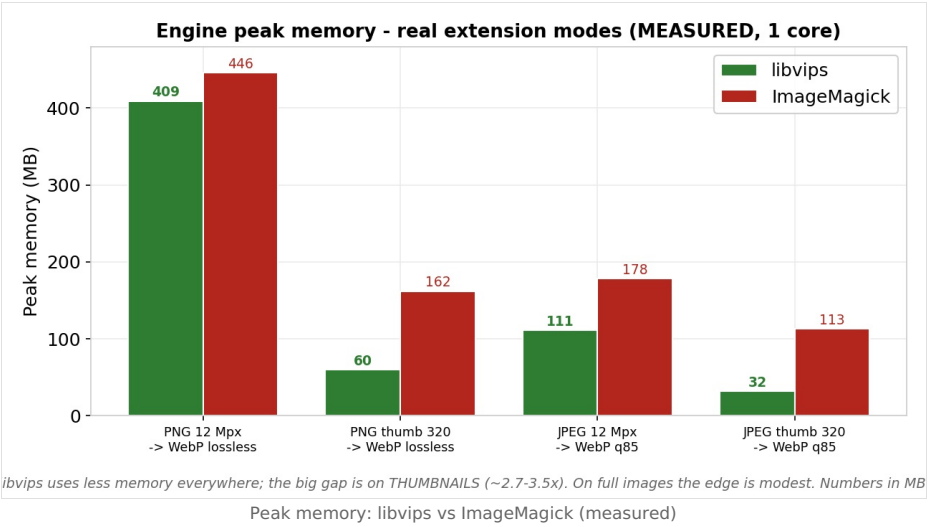
The **engine matters**: GD has no true lossless. At equal quality, **AVIF is not systematically smaller** than WebP: it depends on content and encoder (with GD it only wins at low quality). The AV1 encoder was also missing on the vips side in the test env (automatic WebP fallback). Hence the experimental status.

4.5 GIF & 2nd-pass PNG (disk)

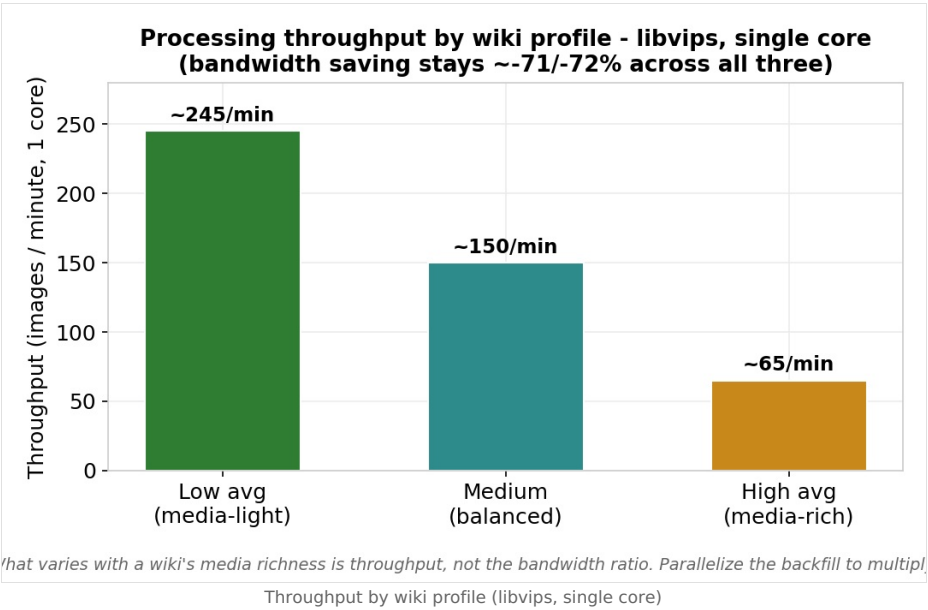


gifsicle: -60 to -66% on GIFs, lossless and animation-safe, for ~20 ms. The **2nd-pass PNG (zopflipng)** only shaves ~-12% of disk... in ~14 s: off-traffic, or prefer oxipng.

4.6 Engine: memory, speed, throughput



Honest reading (measured at the extension's real modes: PNG → lossless WebP, JPEG → q85). libvips uses **less memory everywhere**, especially on **thumbnails** (~2.7–3.5×). On **speed** the advantage is **not** universal: it is large on **JPEG thumbnails** (~40×, thanks to *shrink-on-load*), but on a **big lossless PNG** libvips is **not faster** (even a bit slower: 17 s vs 11 s), because PNG has no reduced-scale decode. Bottom line: vips mainly saves **memory** (and time on JPEG thumbnails).



Throughput varies with media richness (~245 to ~65 images/min on one core); the bandwidth ratio stays ~71/–72%. Parallelize the backfill to multiply throughput.

5. Best practices to avoid slowdowns

5.1 Where does the risk come from?

- **Job queue on web requests.** By default MediaWiki runs jobs at the end of requests (`$wgJobRunRate`): your **visitors** then pay for the encoding.
- **On-the-fly WebP generation at render.** The first view of a cold page encodes the missing WebP files for the thumbnails it uses (bounded by `OnDemandThumbLimit`).
- **Synchronous 2nd pass (if enabled).** zopflipng on a thumbnail can cost hundreds of ms to several seconds (see §4.3).

5.2 First install (especially large wikis)

Option A — all CLI (recommended for large wikis)	Option B — via the queue, controlled
<pre>// LocalSettings.php \$wgVaultTecMediaOptimizerUseJobQueue = false;</pre> Then in a screen: <pre>php maintenance/run.php \ .../optimizeImages.php --batch=200</pre> Off-queue, no visitor impact. Resumable (<code>--start</code>), filterable (<code>--format=gif</code>).	<pre>// LocalSettings.php — 0, no more! \$wgJobRunRate = 0;</pre> Then drain the queue over SSH: <pre>php maintenance/run.php runJobs.php \ --type VaultTecMediaOptimizerOptimizeImage</pre> Leftover jobs: <code>purgeQueue.php</code>.

Do not raise `$wgJobRunRate`: it **worsens** the slowdown (more heavy jobs on visitor requests). For heavy processing the right value is **0**, combined with `runJobs.php` over SSH — or the dedicated CLI script. **The thumbnail zopflipng second pass** also goes through a **job** (since 1.8.2): same rule — process the queue off-request so no visitor pays for it.

- **Pick the engine by volume.** Large media library → **libvips** (memory/speed, §4.4).
- **Tune `OnDemandThumbLimit`** (default 5): low = lighter first renders; 0 = everything via the backfill.
- **Purge caches AT THE END** (wiki + CDN), not during.

5.3 Everyday use

- **GIF:** enable `gifsicle` — clear gain (−66%), negligible cost, animation preserved.
- **2nd-pass PNG:** optional and preferably **oxipng**; keep zopflipng for an off-traffic finishing pass.
- **Long HTTP cache headers** on `images_webp/` (and `images_avif/`): a major, often forgotten lever to actually cut bandwidth.
- **Don't mix queue and CLI** on the same corpus (atomic writes = no corruption, but duplicated work).
- **Permissions:** run scripts as the PHP user, never as `root`.